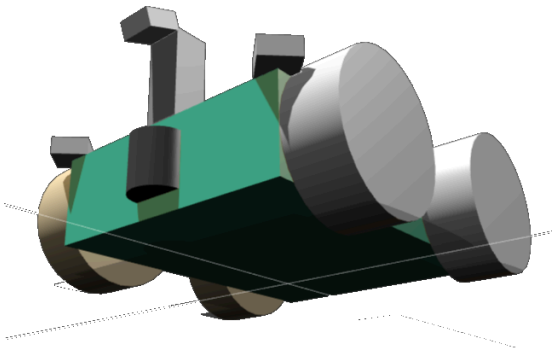
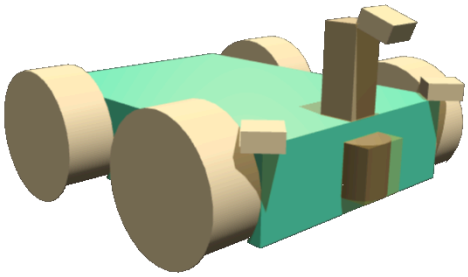
Engineering Physics 353
Machine Learning Project Report

Taiga Momose
Jake Lee





ENPH 353
The University of British Columbia
April 20th, 2026

Table of Contents

1. Introduction

This report is meant to describe and illustrate the methods, architecture, procedures, and problems that arose during the development of an autonomous robot capable of traversing a course while reading text off of signs to decode clues.

1.1. Background

The competition's goal was to develop an autonomous robot that could navigate a course in a virtual ROS Gazebo simulation. The course has multiple NPC's traversing it that the robot must avoid, and 8 clueboards along the pathway with random messages (clues) that the robot must read and publish to the competition environment's ROS `/score_tracker` topic to score points. Given perfect driving, zero collisions with NPC's, and all clueboards read perfectly, the maximum number of points that can be achieved is 57.

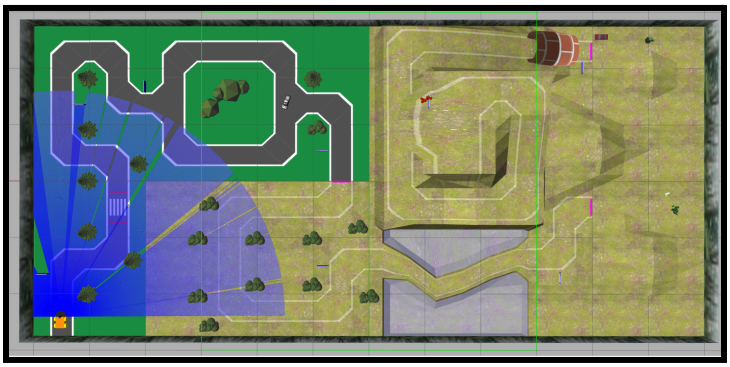


Figure 1.1: Course Layout

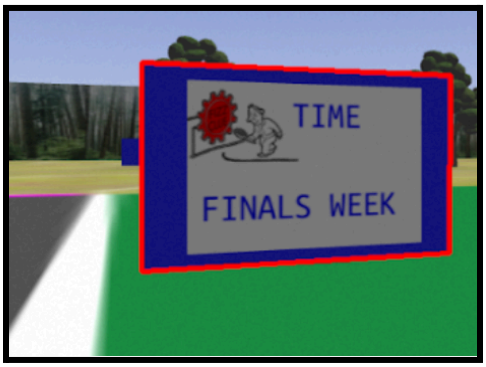


Figure 1.2: Clueboard

1.2. Workload Split

Development of the robot's line following and navigation was led by Taiga, with clueboard detection and reading led by Jake. However, as competition neared, the latter would become a joint-venture, with integration of the two modules being done by Taiga. Google Colab was used to develop the character reading and line following models, simplifying our GitHub repo management to just one `main` branch.

1.3 Software Architecture

The software architecture of the robot went through multiple iterations, with the final product settling on a decoupled driving node and perception node.

The driving node (`353main.py`) is an event-driven finite state machine with a driving state and paused state. It subscribes to the center camera to view the road, the LiDAR mounted at the front to detect NPC's, and the custom topic `/detected_signs` to receive text from clueboards. It then publishes `Twist` commands to `/cmd_vel` to move the robot. The robot starts in the driving state and will transition to the paused state after detecting the pink lines that segment the course. To maintain a clean control node, multiple helper functions and classes are employed. These include `eyes.py` (pink line detection), brains.py (imitation model output), and scanner.py (Baby Yoda detection using LiDAR).

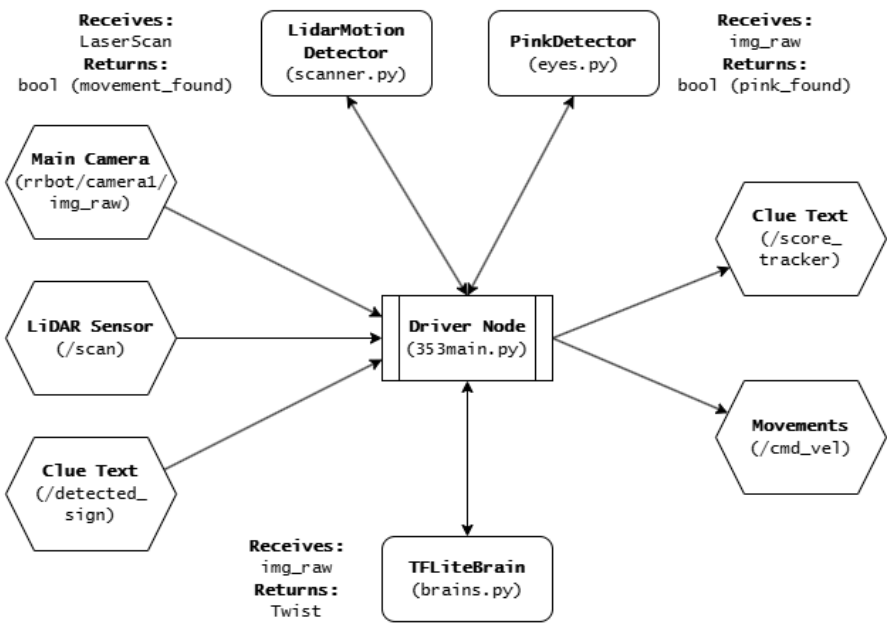


Figure 1.3: Driver Node

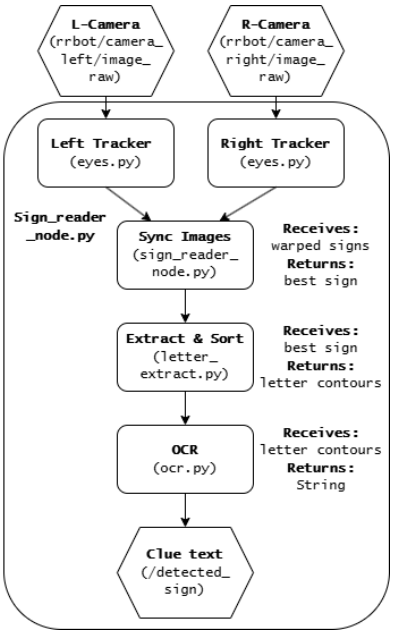


Figure 1.4: Perception node

The perception node (`sign_reader_node.py`) independently handles the sign detection and optical character recognition. It subscribes to the left and right cameras, tracks and returns the best sign images from each using a `SignTracker` object, compares the two, and then extracts and reads the letters from the best sign. The letters are published as a

string to `/detected_signs` that the driving node can receive and publish to `/score_tracker`.

2. Discussion

2.1 Clueboard Detection

Clueboard detection has four main components: border detection, perspective transform, best sign tracker, and character image extraction. Whenever our sign reader node receives an image from our cameras, we first apply border detection to see if there is a clueboard in the frame. If there is, we extract it and apply a perspective transform to make it a perfect image. Since the cameras will capture the same clueboard in multiple consecutive frames, we use the best sign tracker to select the best clueboard image. Finally from the best clueboard, we extract each character.

2.1.1 Border Detection

The first task was to determine how to detect clueboards. After examining the competition environment, we noticed that all the clueboards had blue borders. Therefore, we concluded that using HSV thresholding with contour detection was the simplest yet most robust method for clueboard detection.

From the provided `hsv_threshold.py` script, we found that the lower and upper bounds of (115, 125, 100) and (125, 255, 210) produced the best results for isolating the clueboard. We applied this threshold to each frame we received, which converted it into a binary image. From this binary image, we extracted all white contours using `cv2.findContours`, and kept only the one with the largest area. We also applied an additional threshold requiring the contour to be larger than `min_area` to ensure that the extracted image was sufficiently large. Finally, since we knew that the clueboards were rectangular, we approximated the contour as a polygon and checked whether it had four

vertices. If it did, we identified it as the clueboard and moved on to the perspective transform.

2.1.2 Perspective Transform

The next task was to transform the current clueboard image into a perfect image. The first thing we did was to find the four vertices of the contour we found during the border detection. Here, these vertices have to be ordered from top-left, top-right, bottom-right, and bottom-left since `cv2.getPerspectiveTransform` transforms an image based on four vertices in this order. After ordering the vertices, we performed `cv2.getPerspectiveTransform` to obtain the transformation matrix, and then applied this matrix to our image to get the rectified clueboard. Finally we made a small adjustment by cropping out the blue border from the clueboard image.

2.1.3 Best Sign Tracker

As the robot passes through a clueboard, it captures the same clueboard multiple times. Therefore, we needed a way to pick the best image to use. Our method was to update `best_sign_image` whenever a new clueboard from a new frame had a larger area compared to the current best image. In such cases, we updated both `best_sign_image` and `max_sign_area` to match the new image and moved on to the next frame. We continued doing this until no clueboard was detected for 0.5 seconds. In that case, the clueboard image saved in our `best_sign_image` variable was the best image, and we moved on to the character extraction.

2.1.4 Character Image Extraction

The last task was to extract each letter on the clueboard. A similar approach to detecting the blue border was used here. First, we applied HSV thresholding with the same lower and upper bounds of `(115, 125, 100)` and `(125, 255, 210)`, which left only the “clue” and “value” words. After applying the threshold, we observed that in some cases, two letters were merged together due to some noises. In order to minimize this effect, we applied erosion using a kernel of size (5,5). Then, `cv2.findContours` was used to find all

the contours in the image, and their bounding were found using `cv2.boundingRect`. However, we noticed that erosion alone couldn't completely prevent two letters from merging together. Therefore, we introduced a dynamic splitting logic where the average width of all the bounding boxes were calculated, and any box with its width greater than 1.7 times the average was treated as containing two letters and split into two halves. Next, the average y-coordinates of all boxes were calculated, and boxes with y-coordinates higher than the average were considered to be "clue" whereas y-coordinates lower than the average were considered to be "value". Finally each box was sorted in ascending order of x-coordinates, completing the character extraction.

2.2 Optical Character Recognition

2.2.1 Data Generation

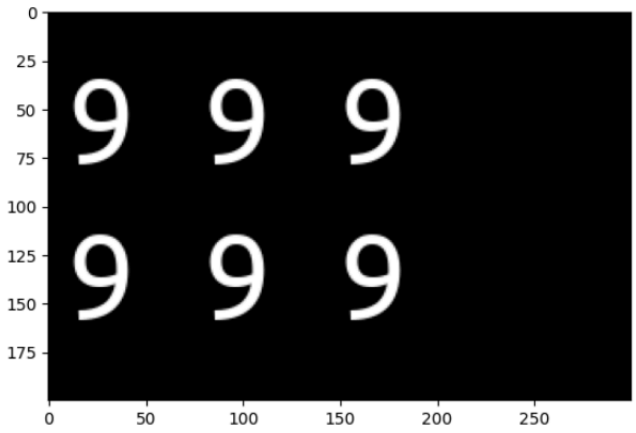


Figure 1.5: Synthetic Character Plate

Instead of modifying the competition environment to display different letters on the clueboard and driving around to collect a dataset, we tried a different approach by manually creating the dataset, which was significantly less time consuming yet still very effective.

By searching through the competition package, we found that the font used for the clueboard was saved in `/usr/share/fonts/truetype/ubuntu/UbuntuMono-R.ttf`. Therefore, using this font, we first wrote a character (starting from A) in white on a black plate, as shown in Figure 1.5. Since we knew what this character was, we also created a label in one-hot vector format.

2.2.2 Image Pre-processing

Randomness was added to the masks by applying augmentations such as blur, zoom, and brightness to better match the competition environment. Finally, we cropped the letters using `cv2.findContours` and `cv2.boundingRect` and saved each image together with its label. This process was iterated through A, B, C, …, 8, and 9. To add more noise to the data set, for each character we then applied two more augmentations: (1) Random translation of 3% in x and y direction, (2) Random zoom of up to 5%.

2.2.3 CNN Architecture

Layer (type)	Output Shape	Param #
augment (Sequential)	(None, 100, 100, 1)	0
conv2d (Conv2D)	(None, 100, 100, 32)	320
max_pooling2d (MaxPooling2D)	(None, 50, 50, 32)	0
conv2d_1 (Conv2D)	(None, 50, 50, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 25, 25, 64)	0
conv2d_2 (Conv2D)	(None, 25, 25, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 12, 12, 128)	0
flatten (Flatten)	(None, 18432)	0
dropout (Dropout)	(None, 18432)	0
dense (Dense)	(None, 128)	2,359,424
dense_1 (Dense)	(None, 36)	4,644

Total params: 2,456,740 (9.37 MB)
Trainable params: 2,456,740 (9.37 MB)
Non-trainable params: 0 (0.00 B)

Figure 1.6: Model Architecture

Figure 1.6 shows a summary of our neural network architecture. An input image of size 100 by 100 is passed through three Conv2D layers with 32, 64, and 128 filters, respectively. A 30% dropout layer is included to reduce overfitting. The final dense layer produces a probability distribution over 36 characters (A-Z and 0-9).

2.2.4. Validation and Training Performance

The model and its training parameters were set using Keras:

```
model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
loss="categorical_crossentropy", metrics=["accuracy"])

cb = [ callbacks.EarlyStopping(monitor="val_accuracy", patience=5,
restore_best_weights=True), callbacks.ReduceLROnPlateau(monitor="val_loss",
factor=0.5, patience=2, verbose=1),
callbacks.ModelCheckpoint("/content/char_cnn.keras", monitor="val_accuracy",
save_best_only=True) ]

history = model.fit(X_train, Y_train, validation_data=(X_val, Y_val),
epochs=30, batch_size=64, callbacks=cb, verbose=1)
```

Appendix H shows a histogram of our training. To reduce overfitting, we implemented `EarlyStopping`, which stopped training if the validation accuracy didn't improve for 5

consecutive epochs. In our case, training stopped after 8 epochs with a validation accuracy of 100%.

2.3 Driving Control

The driver control method used in this project was imitation learning. Frames from the driver node camera would be associated with the `/cmd_vel` `Twist` commands that occurred with them that a CNN would then learn to fit.

2.3.1 Data Collection

High-quality data is essential for good imitation learning. To achieve this, the regular keyboard tele-op commands were replaced by a custom manual controller script that used a PS4 controller (`manual_control.py`) with separate analog joysticks to control steering and linear velocity. To collect the data for a run, another ROS executable called `data_collection.py` is run which subscribes to `/camera1/img_raw` and `/cmd_vel`, and saves the image into a folder and the associated command to a csv file. After a number of runs are collected, the script `prepare_data.py` is used to unify the images and .csv files from each run into one images folder and .csv that can be zipped and uploaded to Google Colab for training.

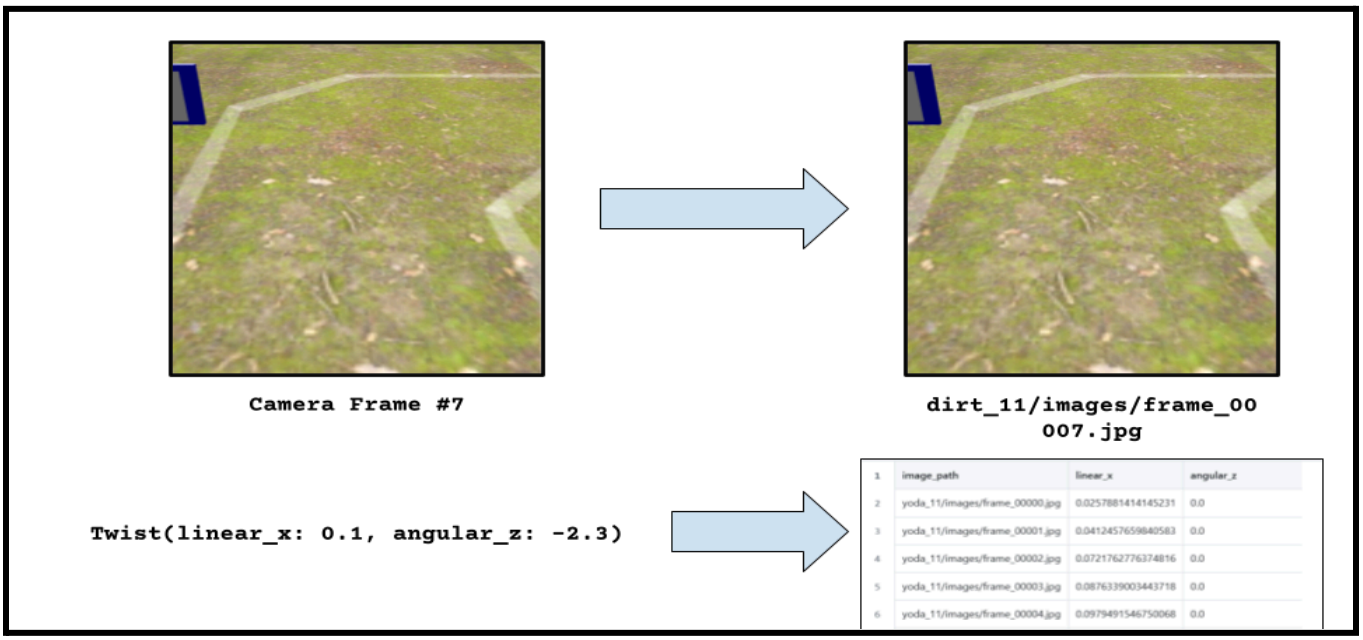


Figure 1.7: Data Collection pipeline

2.3.2 Model Architecture

The model used to train on the prepared data is a Convolutional Neural Network (CNN) based off of NVIDIA's PilotNet architecture (suggested by Gemini, see reference [3]) built using Keras. Before training, the images and commands undergo re-sizing and normalization. The angular_z values are divided by 2.25 (max steering value in manual_control.py) for a normalization of [-1,1] whilst the linear_x values are kept as is (max magnitude of 0.5). The images are resized to 200 x 66 pixels (see Reference X) and the RGB channels are normalized to stay between [0,1].

Layer (type)	Output Shape	Param #
lambda_1 (Lambda)	(None, 66, 200, 3)	0
conv2d_5 (Conv2D)	(None, 31, 98, 24)	1,824
conv2d_6 (Conv2D)	(None, 14, 47, 36)	21,636
conv2d_7 (Conv2D)	(None, 5, 22, 48)	43,248
conv2d_8 (Conv2D)	(None, 3, 20, 64)	27,712
conv2d_9 (Conv2D)	(None, 1, 18, 64)	36,928
flatten_1 (Flatten)	(None, 1152)	0
dropout_2 (Dropout)	(None, 1152)	0
dense_4 (Dense)	(None, 100)	115,300
dropout_3 (Dropout)	(None, 100)	0
dense_5 (Dense)	(None, 50)	5,050
dense_6 (Dense)	(None, 10)	510
dense_7 (Dense)	(None, 2)	22

Total params: 252,230 (985.27 KB)
Trainable params: 252,230 (985.27 KB)
Non-trainable params: 0 (0.00 B)

Figure 1.8: Model Architecture

Feature Extraction:
The model has 5 total convolutional layers. The first three (5x5 kernel, 2x2 stride) are used to capture broad features whilst the last two (3x3 kernel) are used to capture finer details.

Learning Layers:
The feature maps generated by the convolutions are flattened into a 1D vector that is run through dense layers to form connections and dropout layers to prevent overfitting.

2.3.3 Validation and Training Performance

The model training parameters were set using Keras:

```
model.compile(optimizer=optimizers.Adam(learning_rate=0.0001), loss='mse')
history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
batch_size=128, epochs=50, verbose=1)
```

The learning rate and number of epochs did not include any kind of dynamic modification, and were derived from manual testing. The model tended to converge in roughly 10-20 epochs (see Appendix A-D). The data set was also loaded to RAM to significantly expedite the training process.

2.3.4 Troubleshooting & Fine-tuning

After initial training runs, the first models tended to veer off the track and favour right turns. Troubleshooting with Gemini suggested that the training set overrepresented straight sections and right turns due to the long straights on the course. Data from the training set revealed that 75% of frames involved straight driving. To mitigate this, 50% of the straights were removed from the training data. To ensure that bias towards one kind of turn was eliminated, images and their respective `angular_z` commands were also mirrored. Note that stationary data is not included in the data set thanks to `data_collection.py` not recording any frames with 0 velocity.

2.3.6 Model Performance

Four separate imitation learning models were trained. Each one corresponded to a section of the course segmented by the pink lines which, upon detecting, would pause and switch models before resuming. This streamlined the troubleshooting process and proved robust enough for half of all runs to fully traverse the course.

The pavement model required no scaling factors to work reliably, however the models for the post-pavement sections proved to be unstable around turns. To rectify this, the linear velocities were scaled by 0.85, reducing the speed and giving the model more time to react to sharp turns. As for NPC detection of Baby Yoda, the LiDAR is used to detect movement and wait for the Yoda to move out of the line of sight of the robot. Coupled with a 3 second wait upon reaching the Yoda section, the robot is able to completely avoid the NPC.

3. Conclusion

3.1 Competition Performance

In competition, the robot scored 42/57 points. The point of failure was the imitation model for the Yoda section, which lost its path and veered off into the side of the tunnel. However, the sign detection and OCR model managed to read all 6 signs leading up the failure perfectly, and additional points were allocated for technically reaching the tunnel all while avoiding all NPCs.

3.2 Discarded Designs

There were many discarded iterations of the robot. Initially, both sign reading and sign detection was attempted using one 4k front mounted camera, however, this proved unreliable for reading signs especially on tight turns. To view more of the road, the driving camera was also placed on a 5cm tall mast and tilted down. This modification was a holdover from initial attempts to implement PID steering, but was kept during the switch to imitation learning since it still provided an improved view of the environment.
The decoupled driving and perception node structure was initially a single driving node that handled driving and perception simultaneously. However, this resulted in race-conditions that delayed the imitation models from reacting to images in time, destroying model performance and the Real Time Factor of the Gazebo environment.

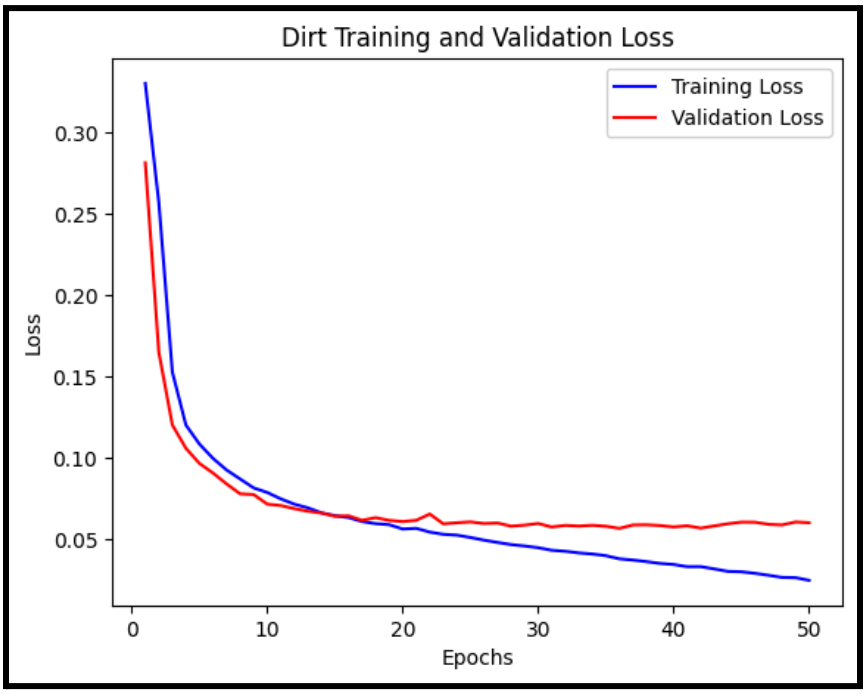
3.3 Improvements

For future improvements, a more robust methodology of collecting training data should be implemented. For the competition, most of the training data consisted of clean runs that were split into validation and training data by a random 20/80 split. More snippets of run recoveries should be added to the training set to allow the robot to learn how to recover from errors. Attempts to include this kind of data were made, but time restricted the amount of progress on this front.

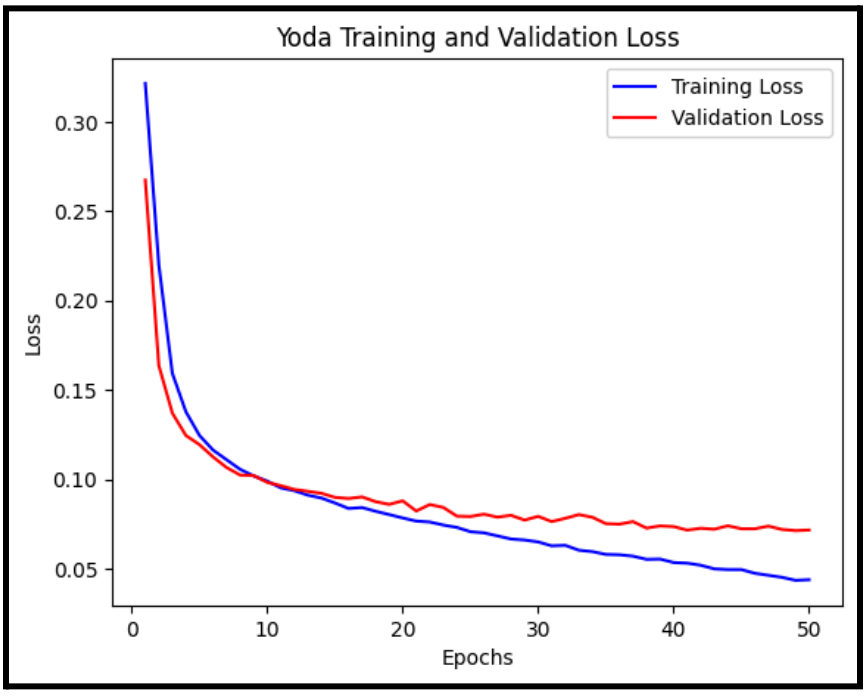
4. References

- [1] https://docs.opencv.org/4.13.0/ [OpenCV2 docs]
- [2] https://wiki.ros.org/ [Used as a ROS help and lookup sheet]
- [3] https://arxiv.org/pdf/1604.07316 [Suggested by Gemini, CNN PilotNet Architecture]
- [4] Google Gemini (browser) [Used as a debugging, coding, and research tool]
- [5] https://viewer.robotsfan.com/ [Used to visualize robot changes before deploying]

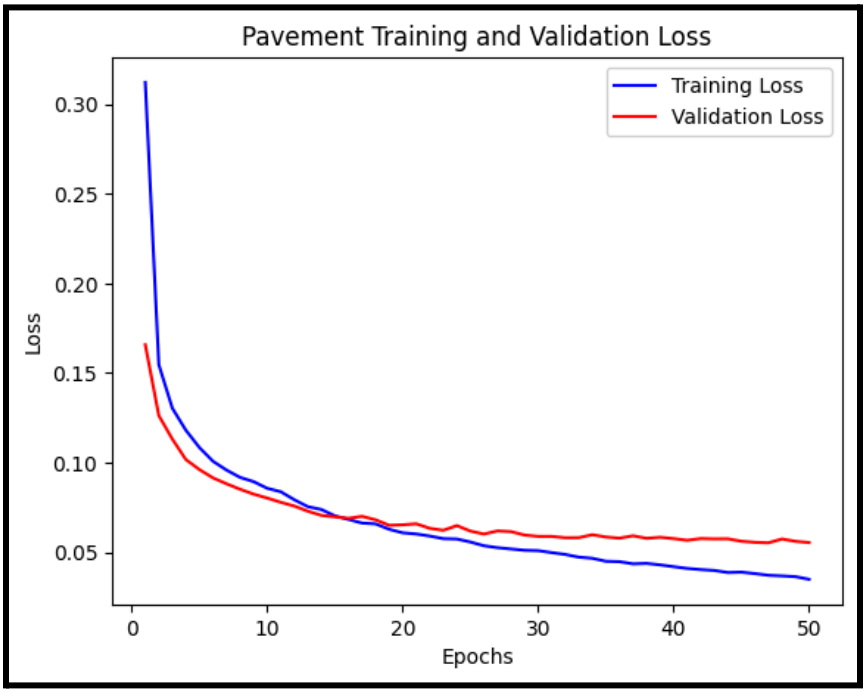
A. Appendix



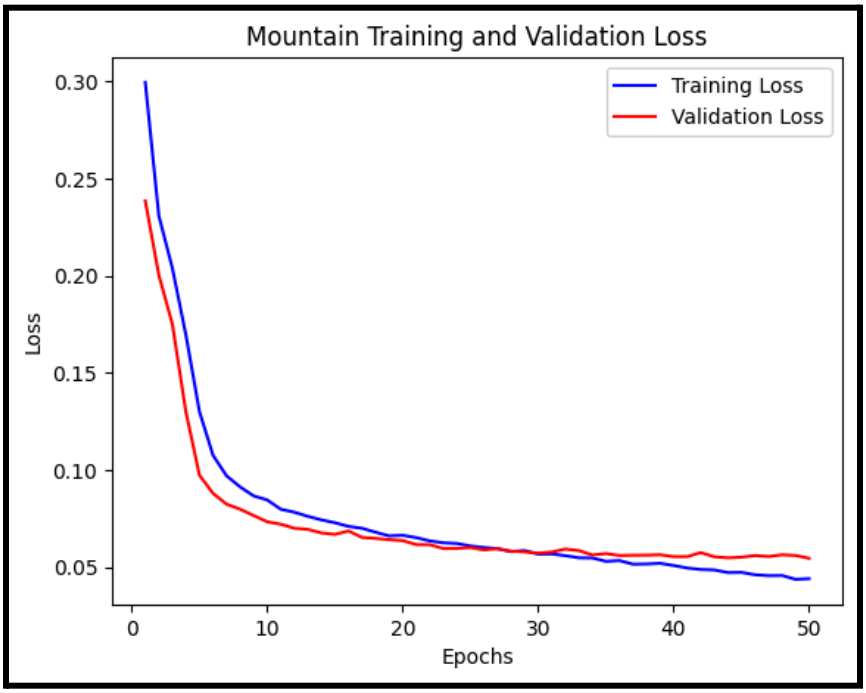
Appendix A: Dirt Imitation Model Training Histogram



Appendix B: Yoda Imitation Model Training Histogram

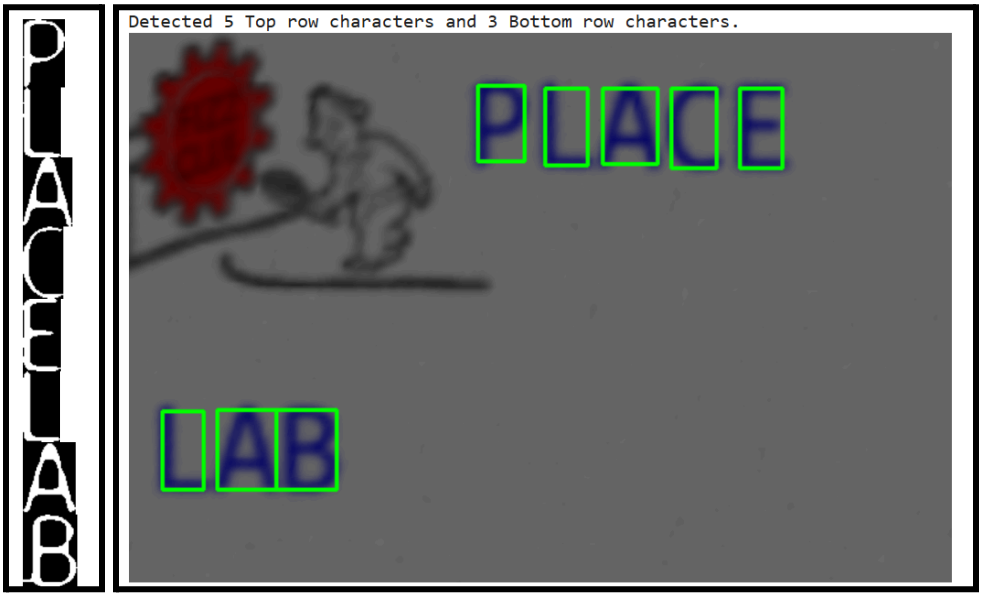


Appendix C: Pavement Imitation Model Training Histogram

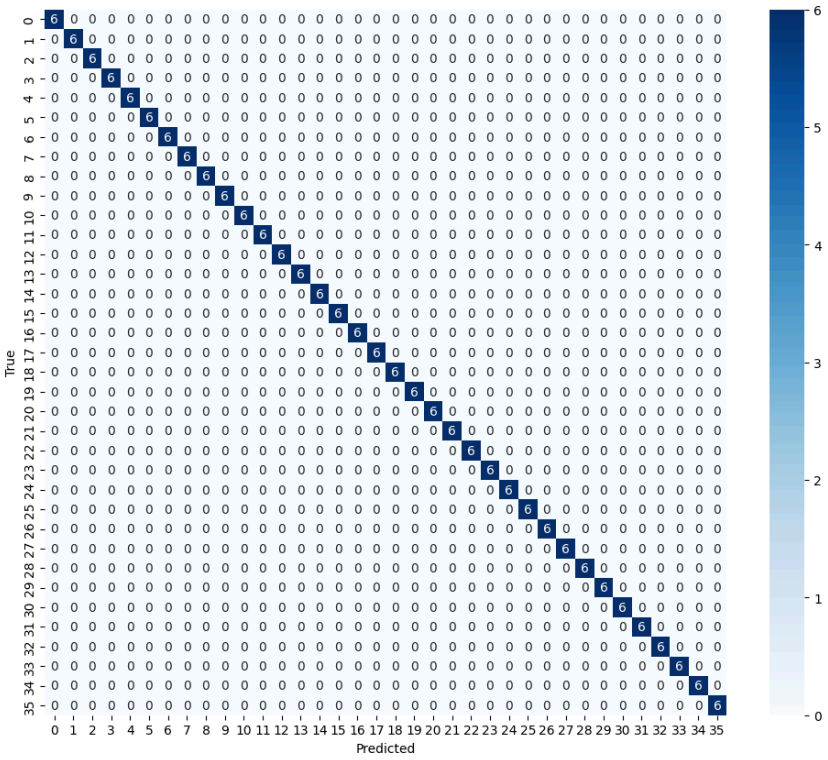


Appendix D: Mountain Imitation Model Training Histogram

Appendix E: Post-Processed Clueboard



Appendix F: Extracted Letters & Contour Bounding Boxes



Appendix G: Confusion Matrix of the OCR Model (overfitting)

Appendix H: OCR Model Training Histogram